Universidad de El Salvador

Facultad de Ingeniería y Arquitectura

Educación en Línea

Ingeniería en Desarrollo de Software

Asignatura:

Desarrollo y Reutilización de Software

Grupo Teórico:

Grupo Teórico 1

Proyecto – DRS135 – Documento de enfoque utilizado

Integrantes:

Grupo 1 – Tema 5		
No	**Nombre completo**	**Correo institucional**
1	Alejandra Michelle Mejía Rivas	mr22035@ues.edu.sv
2	Christopher Alexis Velásquez Aguilar	va22020@ues.edu.sv
3	Iván Eduardo López Tobar	lt22009@ues.edu.sv
4	Josué Daniel Rodríguez Yanes	ry22001@ues.edu.sv
5	Irvin Elías Torres Merlos	tm22012@ues.edu.sv
6	Carlos Alberto Ávila González	ag22029@ues.edu.sv
7	José Emmanuel García Rodríguez	gr22081@ues.edu.sv

Fecha:

25/10/2026

1. Enfoque Utilizado

1.1 Filosofía de Diseño

Se adoptó un enfoque basado en los siguientes principios:

Separación de responsabilidades (SoC): Cada capa del sistema tiene una responsabilidad bien definida y no se mezclan lógica de presentación, negocio y persistencia.

Reutilización por frameworks: En lugar de implementar desde cero un servidor HTTP, manejo de rutas, serialización JSON y ciclo de vida de la aplicación, se reutilizó el ecosistema completo de Spring Boot.

Patrones de diseño probados: Se aplicaron patrones como MVC, Singleton e Initialization-on-Demand Holder (variante thread-safe de Singleton) para resolver problemas recurrentes de diseño.

- **Mínima dependencia, máxima cohesión:** El proyecto depende únicamente de spring-boot-starter-webmvc (que incluye Jackson para JSON), evitando dependencias innecesarias.

1.2 Metodología de Desarrollo

Se identificaron los requisitos funcionales: CRUD de usuarios.

Se seleccionó Spring Boot como framework reutilizable que provee MVC, inyección de dependencias y un servidor embebido.

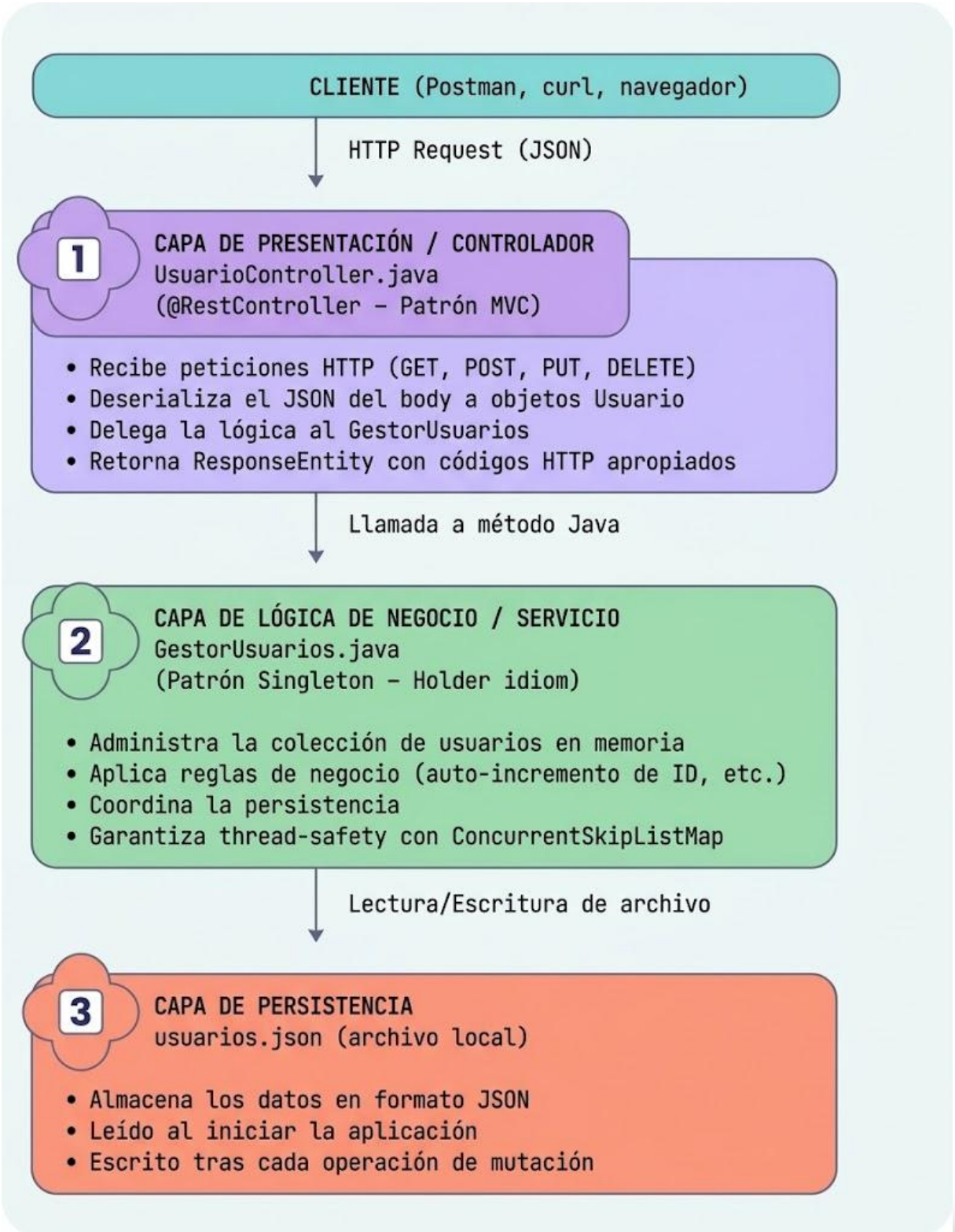
Se diseñó la arquitectura en tres capas: Modelo, Controlador y Gestor (Manager/Service).

Se implementó el patrón Singleton para centralizar la gestión de datos.

1. Se utilizó Jackson (incluido en Spring Boot) como componente reutilizable para la serialización/deserialización JSON.

2. Arquitectura del Sistema

El sistema sigue una **arquitectura de tres capas** inspirada en el patrón MVC, adaptada al contexto de una API REST:


CLIENTE (Postman, curl, navegador)
HTTP Request (JSON)
1
CAPA DE PRESENTACIÓN / CONTROLADOR
UsuarioController.java
(@RestController – Patrón MVC)
• Recibe peticiones HTTP (GET, POST, PUT, DELETE)
• Deserializa el JSON del body a objetos Usuario
• Delega la lógica al GestorUsuarios
• Retorna ResponseEntity con códigos HTTP apropiados
Llamada a método Java
2
CAPA DE LÓGICA DE NEGOCIO / SERVICIO
GestorUsuarios.java
(Patrón Singleton – Holder idiom)
• Administra la colección de usuarios en memoria
• Aplica reglas de negocio (auto-incremento de ID, etc.)
• Coordina la persistencia
• Garantiza thread-safety con ConcurrentSkipListMap
Lectura/Escritura de archivo
3
CAPA DE PERSISTENCIA
usuarios.json (archivo local)
• Almacena los datos en formato JSON
• Leído al iniciar la aplicación
• Escrito tras cada operación de mutación

Capa Modelo
La clase Usuario es un **POJO (Plain Old Java Object)** que representa la entidad de dominio:

Campo	Tipo	Descripción
id	int	Identificador único auto-generado
nombre	String	Nombre completo del usuario
correo	String	Correo electrónico

3. Patrones de Diseño Implementados

3.1 Patrón MVC (Model-View-Controller)

¿Dónde se aplica? En la estructura completa del proyecto, proporcionada por Spring Boot.

Componente MVC	Implementación en el Proyecto	Archivo
Model	Clase Usuario — Representa los datos del dominio	model/Usuario.java
Controller	Clase UsuarioController — Maneja las peticiones HTTP y delega al servicio	controller/UsuarioController.java
View	Las respuestas JSON (en una API REST, la "vista" es la representación JSON serializada por Jackson)	Generado automáticamente por Spring

¿Cómo funciona?

El **Controller** recibe una petición HTTP (p.ej., POST /api/usuarios con un body JSON).

Spring Boot (a través de Jackson) deserializa automáticamente el JSON en un objeto **Model** (Usuario).

El Controller delega la operación al **GestorUsuarios** (capa de servicio/lógica).

1. El resultado se serializa automáticamente a JSON y se devuelve como respuesta HTTP (**View**).

Beneficio para la reutilización: El patrón MVC permite modificar o reemplazar cualquier capa de forma independiente. Por ejemplo, se podría cambiar la persistencia de archivo JSON a una base de datos relacional sin modificar el Controller ni el Modelo.

3.2 Patrón Singleton (Initialization-on-Demand Holder)

¿Dónde se aplica? En la clase GestorUsuarios.

```
// Constructor privado: impide instanciación externa
private GestorUsuarios() {
    cargarDesdeArchivo();
}

// Clase estática interna (Holder) — la instancia se crea
// solo cuando se accede por primera vez a getInstance()
private static class Holder {
    private static final GestorUsuarios INSTANCIA = new GestorUsuarios();
}

// Punto de acceso global
public static GestorUsuarios getInstance() {
    return Holder.INSTANCIA;
}
```

¿Por qué se usa esta variante?

Aspecto	Singleton clásico	Holder idiom (este proyecto)
Thread-safety	Requiere synchronized o volatile	Thread-safe por diseño del ClassLoader de Java
Rendimiento	Penalización por sincronización	Sin overhead — la JVM inicializa la clase Holder una sola vez

Aspecto	Singleton clásico	Holder idiom (este proyecto)
Inicialización	Eager o lazy con doble verificación	Lazy natural — se crea solo al invocar getInstance()

Beneficio para la reutilización: Garantiza una única instancia del gestor de datos en toda la aplicación, evitando inconsistencias cuando múltiples controladores o componentes acceden a la misma colección de usuarios.

3.3 Patrón Observer (Implícito vía Spring Boot)

Si bien no se implementó un Observer explícito con interfaces Observer/Observable, Spring Boot aplica internamente este patrón a través de:

Listeners de eventos del contexto de aplicación: Spring dispara eventos como ApplicationReadyEvent, ContextRefreshedEvent, etc., y los beans pueden suscribirse con @EventListener.

Modelo de anotaciones reactivas: Las anotaciones @GetMapping, @PostMapping, etc., funcionan como "suscripciones" a eventos HTTP específicos: el DispatcherServlet observa las peticiones entrantes y notifica al controlador apropiado.

- **Autoconfiguración:** Spring Boot observa las dependencias presentes en el classpath y auto-configura los beans necesarios (p.ej., detecta spring-boot-starter-webmvc y configura automáticamente el servidor Tomcat embebido y el DispatcherServlet).

4. Reutilización de Código y Componentes

4.1 Framework Spring Boot (Reutilización de Terceros)

Spring Boot es un **framework reutilizable** que proporciona la mayor parte de la infraestructura de la aplicación:

Componente Reutilizado	Qué provee	Sin Spring Boot requeriría...
Tomcat embebido	Servidor HTTP completo	Configurar manualmente un servidor HTTP
DispatcherServlet	Enrutamiento de peticiones a controladores	Implementar un router propio

Componente Reutilizado	Qué provee	Sin Spring Boot requeriría...
Jackson	Serialización/deserialización JSON	Escribir parsers JSON manualmente
Spring IoC Container	Gestión del ciclo de vida de componentes	Instanciar y conectar objetos manualmente
spring-boot-starter-webmvc	Dependencia única que incluye todo lo anterior	Gestionar múltiples dependencias individuales
Maven (wrapper)	Sistema de build y gestión de dependencias	Compilar y empaquetar manualmente

Impacto cuantitativo: El proyecto tiene aproximadamente **210 líneas de código propio**. Sin el framework, una funcionalidad equivalente requeriría **miles de líneas** de código para el servidor HTTP, enrutamiento, serialización y gestión de ciclo de vida.

4.2 Jackson ObjectMapper (Reutilización de Biblioteca)

La clase GestorUsuarios reutiliza ObjectMapper de Jackson para la persistencia:

```
private final ObjectMapper mapper = JsonMapper.builder()
        .enable(SerializationFeature.INDENT_OUTPUT)
        .build();

// Lectura: JSON → Lista de objetos Java
List<Usuario> lista = mapper.readValue(archivo, new
TypeReference<List<Usuario>>() {});

// Escritura: Objetos Java → JSON
mapper.writeValue(new File(ARCHIVO), listar());
```

Esto permite convertir entre objetos Java y JSON sin escribir lógica de serialización manual.

4.3 Reutilización Interna dentro del Proyecto

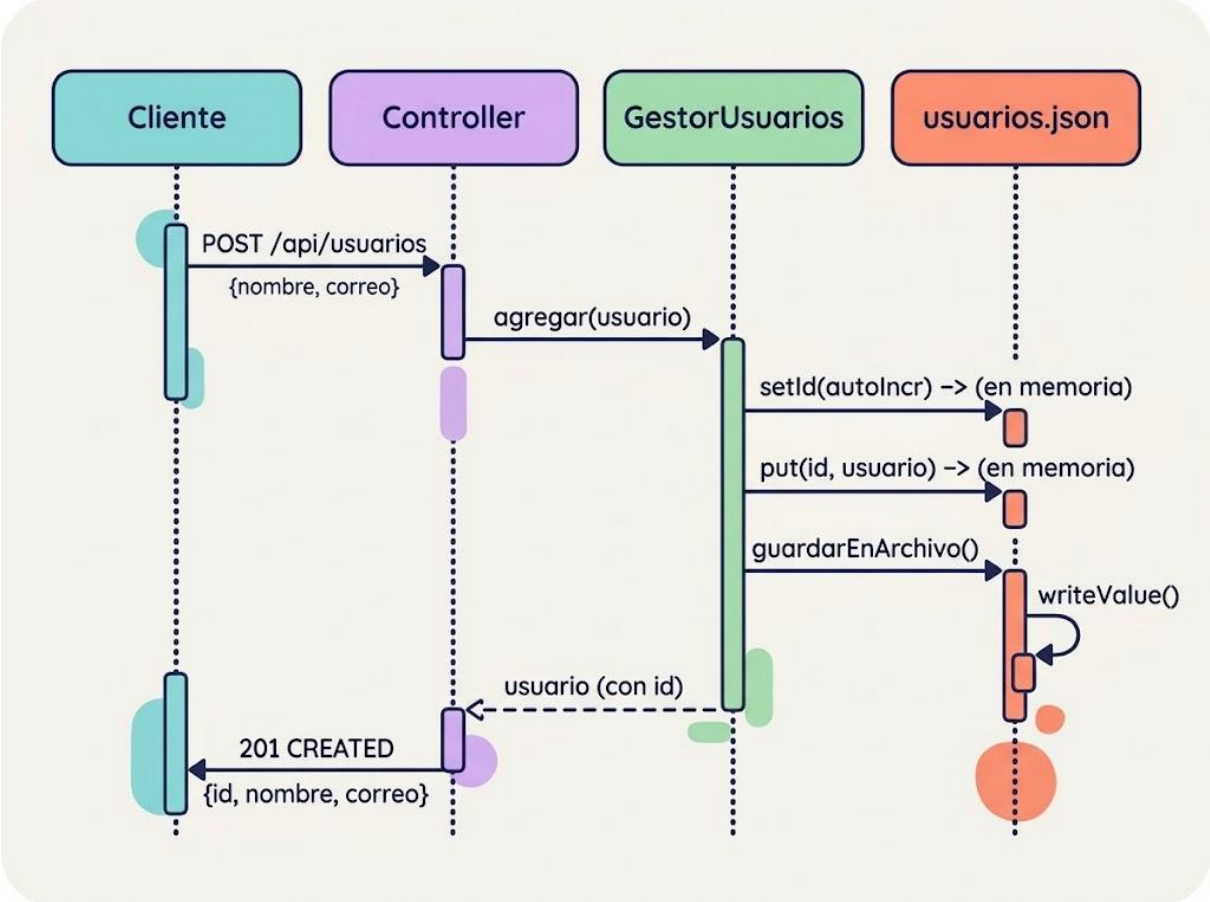
Componente	Reutilizado por	Cómo
GestorUsuarios.getInstance()	UsuarioController	El controlador obtiene siempre la misma instancia del gestor
GestorUsuarios.listar()	guardarEnArchivo()	El método de persistencia reutiliza listar() para obtener los datos a serializar
Clase Usuario (POJO)	Controller, GestorUsuarios, Jackson	El mismo modelo es usado para recibir datos HTTP, procesarlos y persistirlos
ConcurrentSkipListMap (JDK)	GestorUsuarios	Colección thread-safe de la biblioteca estándar de Java, reutilizada en lugar de implementar sincronización manual
AtomicInteger (JDK)	GestorUsuarios	Contador atómico del JDK para generar IDs únicos sin necesidad de sincronización explícita

4.4 Reutilización de Maven Wrapper

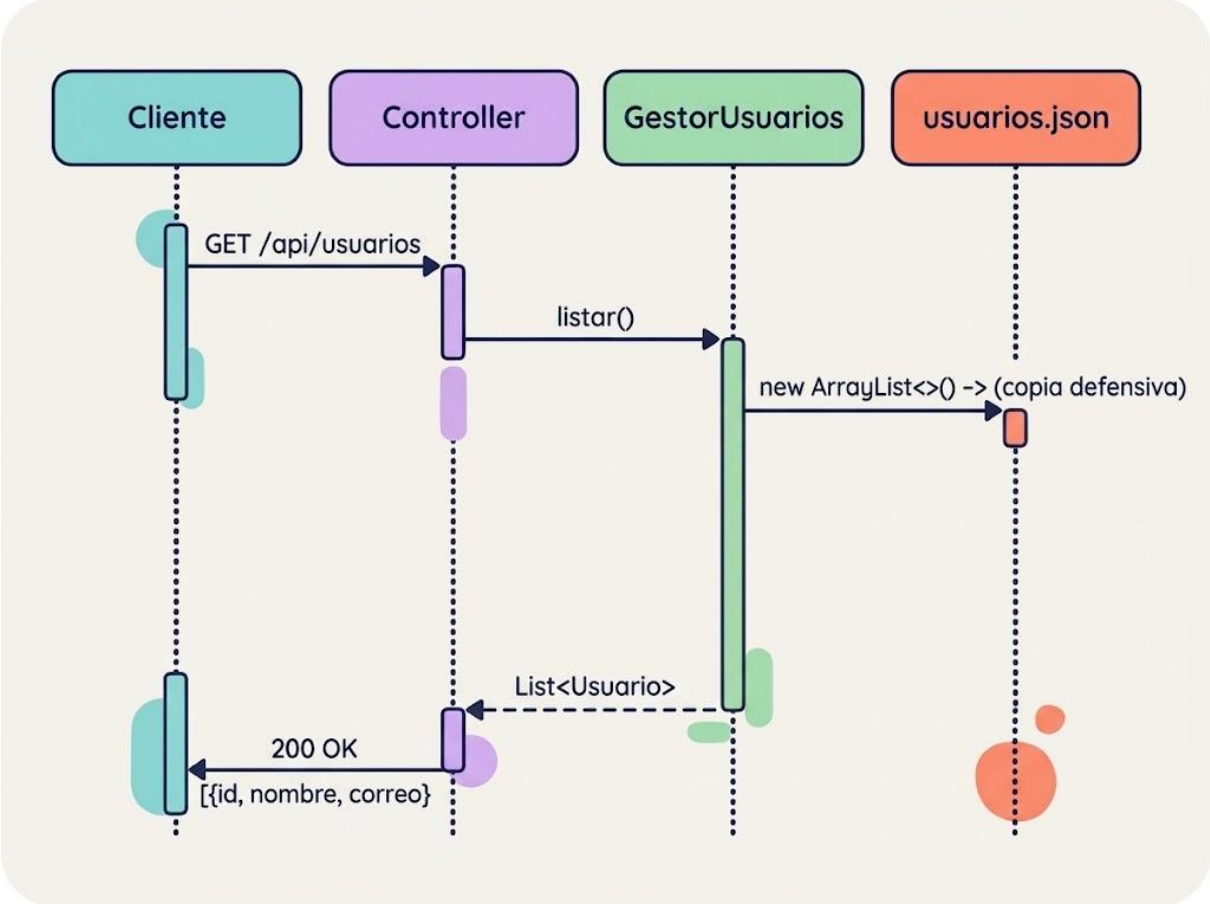
El proyecto incluye mvnw y mvnw.cmd, que son scripts reutilizables generados por el Maven Wrapper. Estos permiten compilar y ejecutar el proyecto **sin necesidad de instalar Maven localmente**, garantizando reproducibilidad entre diferentes entornos de desarrollo.

5. Interacción entre los Componentes

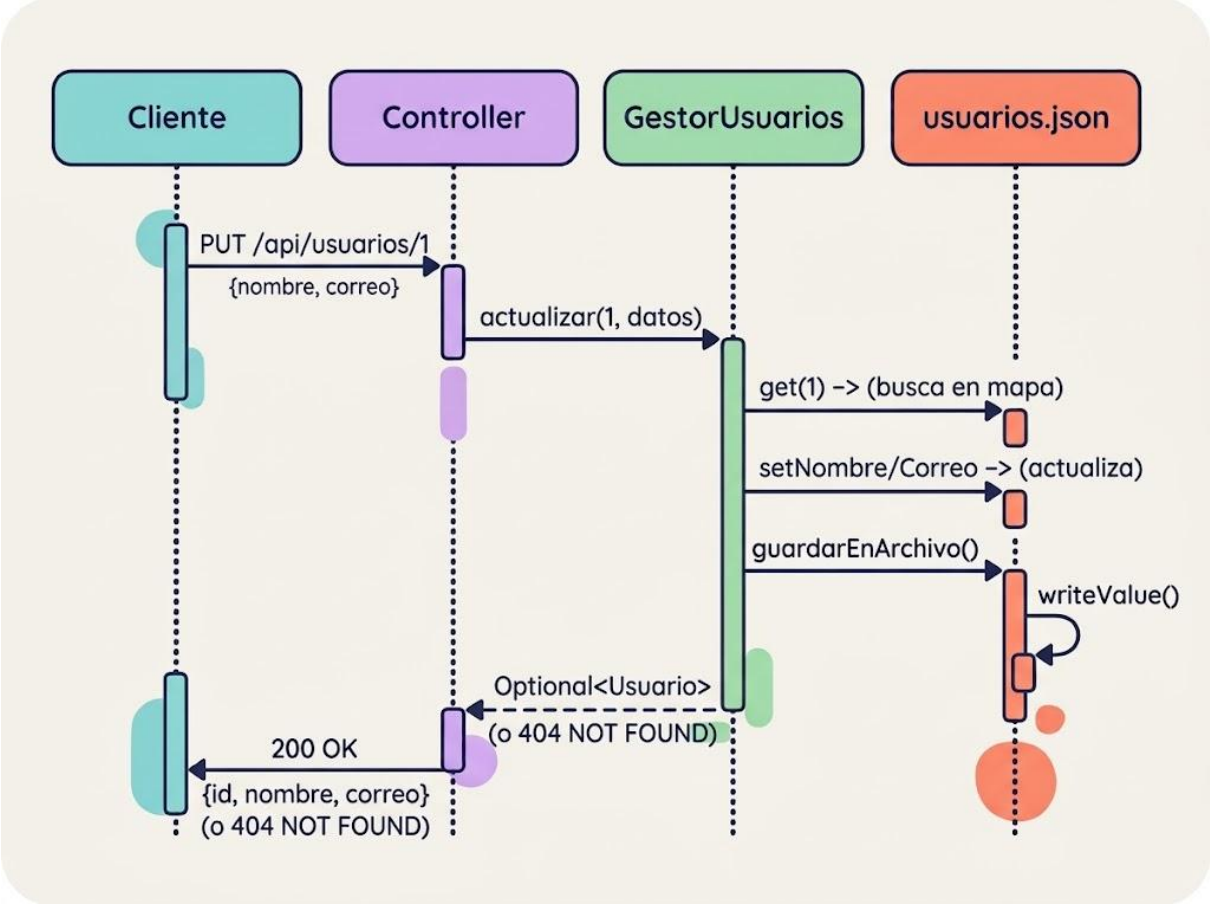
5.1 Diagrama de Flujo — Crear un Usuario (POST)



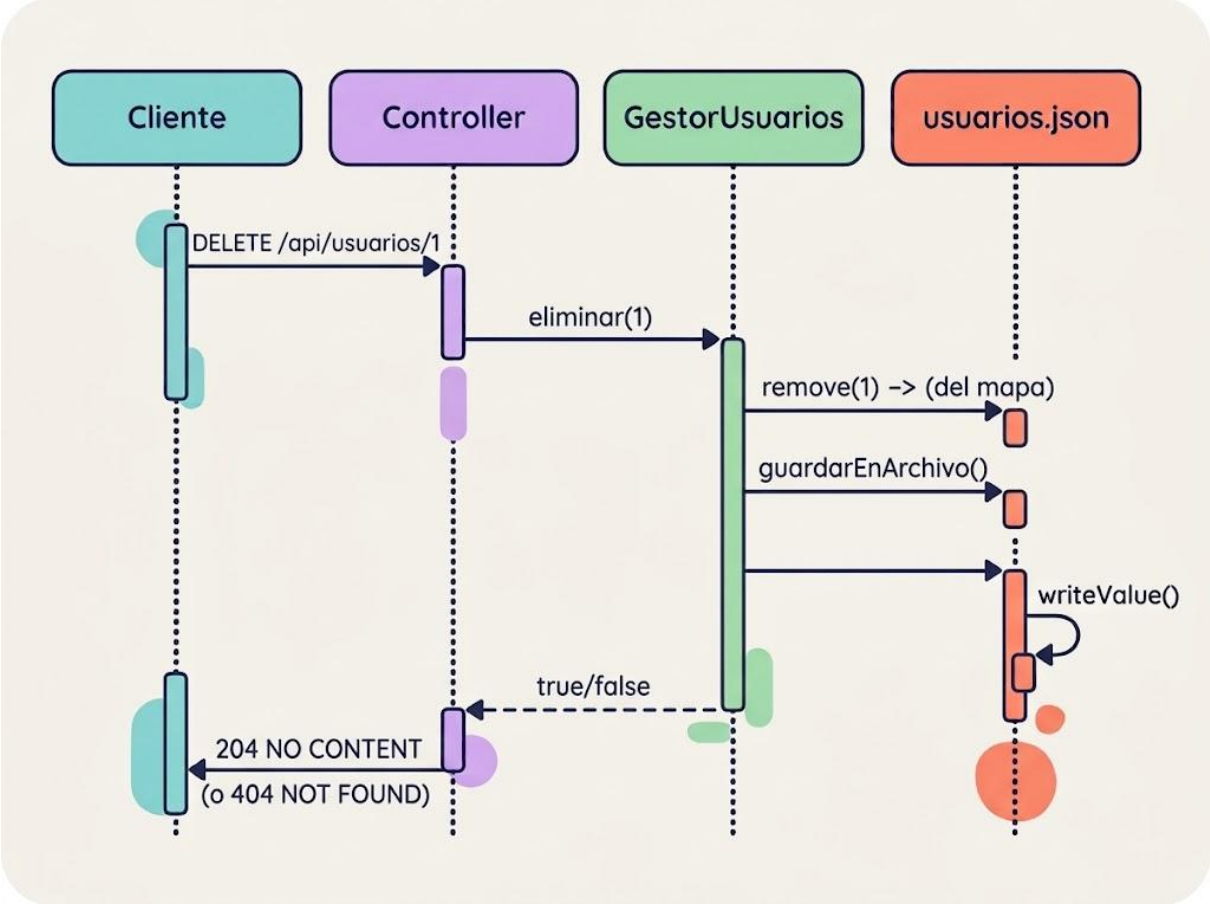
5.2 Diagrama de Flujo — Listar Usuarios (GET)



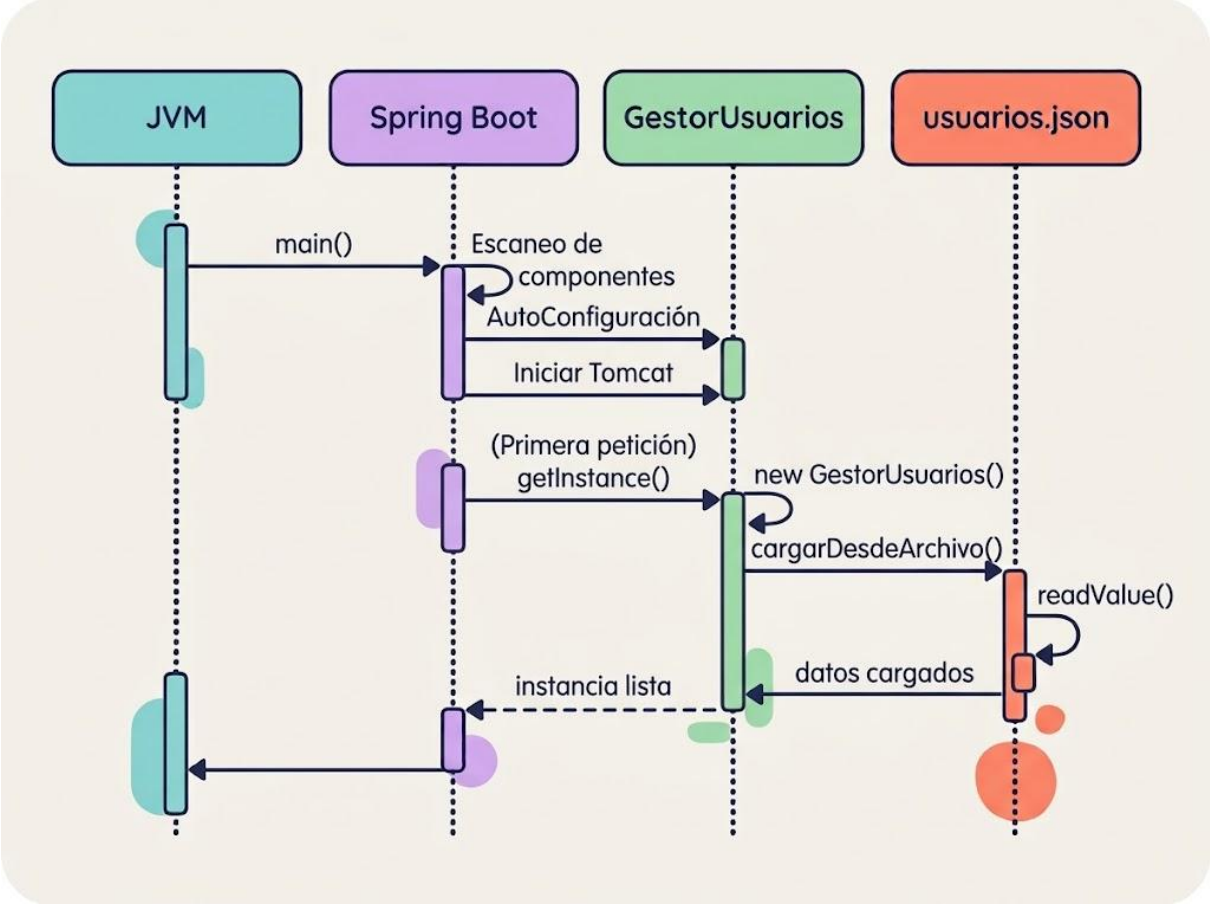
5.3 Diagrama de Flujo — Actualizar un Usuario (PUT)



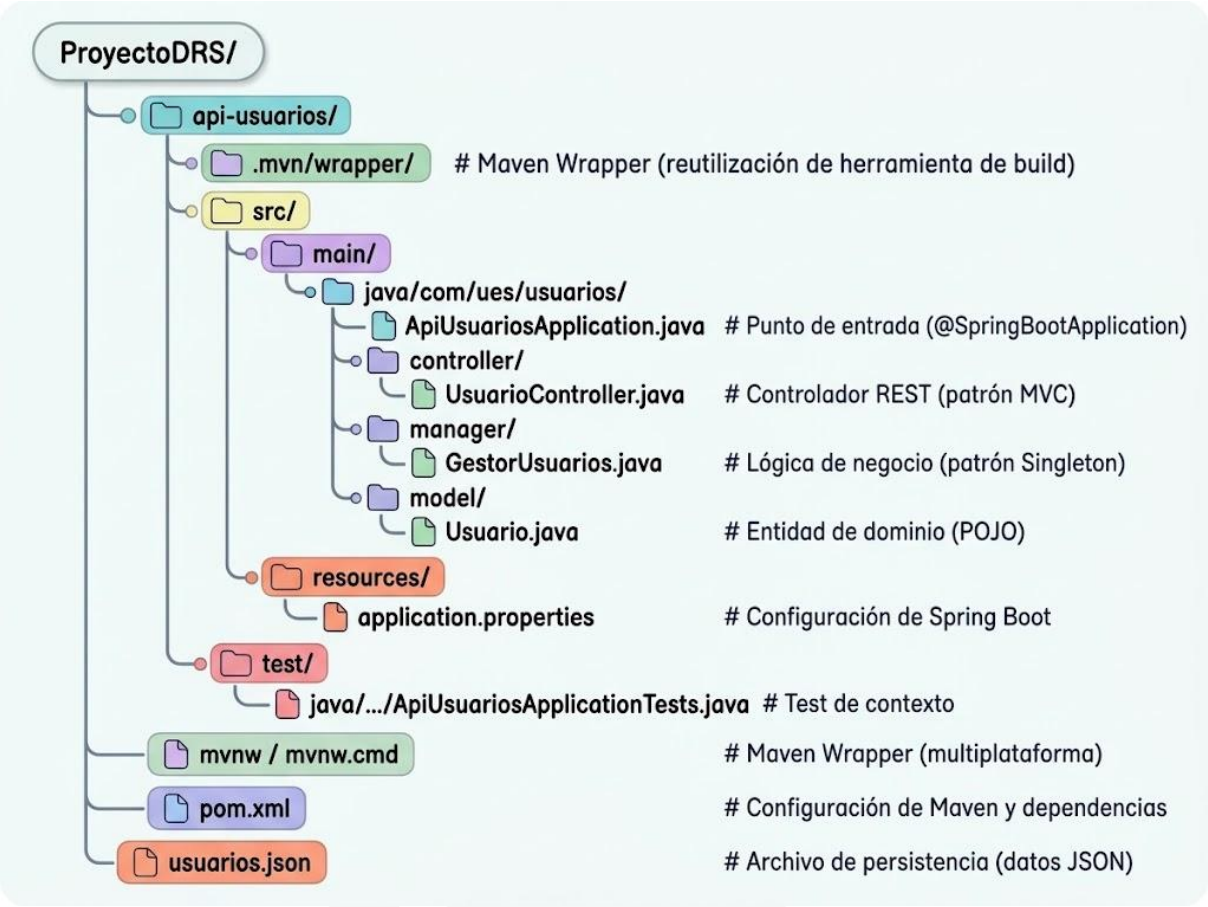
5.4 Diagrama de Flujo — Eliminar un Usuario (DELETE)



5.5 Ciclo de Vida de la Aplicación



6. Estructura del Proyecto



7. Decisiones de Diseño

7.1 ¿Por qué Spring Boot en lugar de código desde cero?

Criterio	Sin Framework	Con Spring Boot
Tiempo de desarrollo	Semanas	Horas
Líneas de código	Miles	~210 (código propio)
Manejo de HTTP	Implementación manual con HttpServer	Automático con @RestController
Serialización JSON	Parser manual o dependencia directa	Incluido con Jackson
Mantenibilidad	Baja (todo acoplado)	Alta (capas independientes)

Decisión: Se optó por Spring Boot para demostrar cómo la **reutilización de frameworks** reduce drásticamente el esfuerzo de desarrollo y produce código más robusto.

7.2 ¿Por qué persistencia en archivo JSON y no en base de datos?

Simplicidad: No requiere instalar ni configurar un motor de base de datos.

Portabilidad: El archivo usuarios.json es legible por humanos y transportable.

Foco en patrones: El objetivo del proyecto es demostrar patrones de diseño, no la capa de datos. Usar un archivo JSON mantiene la atención en la arquitectura.

- **Fácil migración:** Gracias a la separación de capas, reemplazar la persistencia por una base de datos (JPA/Hibernate) solo requeriría modificar GestorUsuarios, sin tocar el controlador ni el modelo.

7.3 ¿Por qué Singleton con Holder en lugar de @Service de Spring?

El uso explícito del patrón Singleton con la técnica del Holder idiom se eligió como **decisión pedagógica** para:

1. Demostrar la implementación manual del patrón Singleton como indica la rúbrica.
2. Mostrar una variante thread-safe sin necesidad de synchronized en el método getInstance().
3. Ilustrar que Spring en sí mismo gestiona sus beans como Singletons por defecto (scope singleton), pero implementarlo manualmente demuestra comprensión profunda del patrón.

7.4 ¿Por qué ConcurrentSkipListMap y AtomicInteger?

ConcurrentSkipListMap: Proporciona un mapa ordenado y thread-safe sin necesidad de sincronización explícita para operaciones individuales. Esto reutiliza las clases concurrentes del JDK en lugar de implementar manejo manual de hilos.

AtomicInteger: Genera IDs únicos de forma atómica, reutilizando la API de concurrencia del JDK.

- **synchronized en actualizar() y eliminar():** Estas operaciones requieren atomicidad compuesta (leer + modificar + persistir), por lo que se utiliza sincronización explícita solo donde es necesario.

7.5 ¿Por qué Java y no C#?

Aunque la rúbrica sugiere C# como opción principal, Java fue una alternativa válida contemplada. Ambos lenguajes son orientados a objetos, soportan los mismos patrones de diseño, y tienen ecosistemas equivalentes:

Concepto	C# / .NET	Java / Spring Boot
Framework MVC	ASP.NET Core MVC	Spring Boot Web MVC
Inyección de dependencias	Built-in DI	Spring IoC Container
Serialización JSON	System.Text.Json	Jackson
Patrón Singleton	Mismo patrón	Mismo patrón
API REST	Controllers con [ApiController]	Controllers con @RestController

Los conceptos de reutilización de software son **independientes del lenguaje**.

8. Escalabilidad y Mantenibilidad

8.1 Puntos de Extensión

Gracias a la arquitectura modular, el sistema puede extenderse fácilmente:

Extensión	Qué cambiar	Qué NO cambiar
Agregar campo telefono a Usuario	Solo Usuario.java	Controller, GestorUsuarios
Migrar a base de datos (MySQL, PostgreSQL)	Solo GestorUsuarios (reemplazar lectura/escritura de archivo)	Controller, Usuario
Agregar validaciones	Agregar anotaciones @Valid, @NotBlank en Controller y Usuario	GestorUsuarios
Agregar autenticación	Agregar spring-boot-starter-security y configurar filtros	Controller, GestorUsuarios, Usuario
Agregar otro recurso (p.ej., Producto)	Crear nuevos ProductoController, GestorProductos, Producto	Nada existente se modifica
Agregar patrón Observer explícito	Crear interfaces Observer/Subject, implementar notificaciones en GestorUsuarios	Controller, Usuario

8.2 Principios SOLID Aplicados

Principio	Aplicación en el Proyecto
S — Responsabilidad Única	Cada clase tiene una sola razón para cambiar: Usuario = datos, Controller = HTTP, GestorUsuarios = lógica + persistencia
O — Abierto/Cerrado	Se pueden agregar nuevos endpoints sin modificar los existentes
L — Sustitución de Liskov	Usuario podría ser extendido por subclases sin romper el sistema
I — Segregación de Interfaces	El Controller solo expone los métodos HTTP necesarios
D — Inversión de Dependencias	El Controller depende de GestorUsuarios a través de su interfaz pública, no de la implementación interna

9. Conclusiones

La reutilización de software es clave para la productividad: Spring Boot permitió construir una API REST completa con solo ~210 líneas de código propio, reutilizando miles de líneas de código del framework.

Los patrones de diseño resuelven problemas recurrentes:

MVC separó las responsabilidades y facilitó la mantenibilidad.

Singleton garantizó consistencia de datos y evitó instancias duplicadas del gestor.

Observer (implícito en Spring) permite que el sistema reaccione a eventos de forma desacoplada.

La modularidad facilita la evolución: Cada capa puede ser modificada, extendida o reemplazada de forma independiente, lo cual es esencial para proyectos que crecen con el tiempo.

La reutilización opera en múltiples niveles:

Nivel de framework: Spring Boot, Maven.

Nivel de biblioteca: Jackson, clases concurrentes del JDK.

Nivel de código propio: La clase Usuario es reutilizada por todas las capas; GestorUsuarios centraliza la lógica para cualquier consumidor.

Los conceptos son transferibles: Los patrones y principios aplicados en Java son idénticos a los que se aplicarían en C# con ASP.NET Core, demostrando que la reutilización de software es un concepto universal e independiente de la tecnología específica.